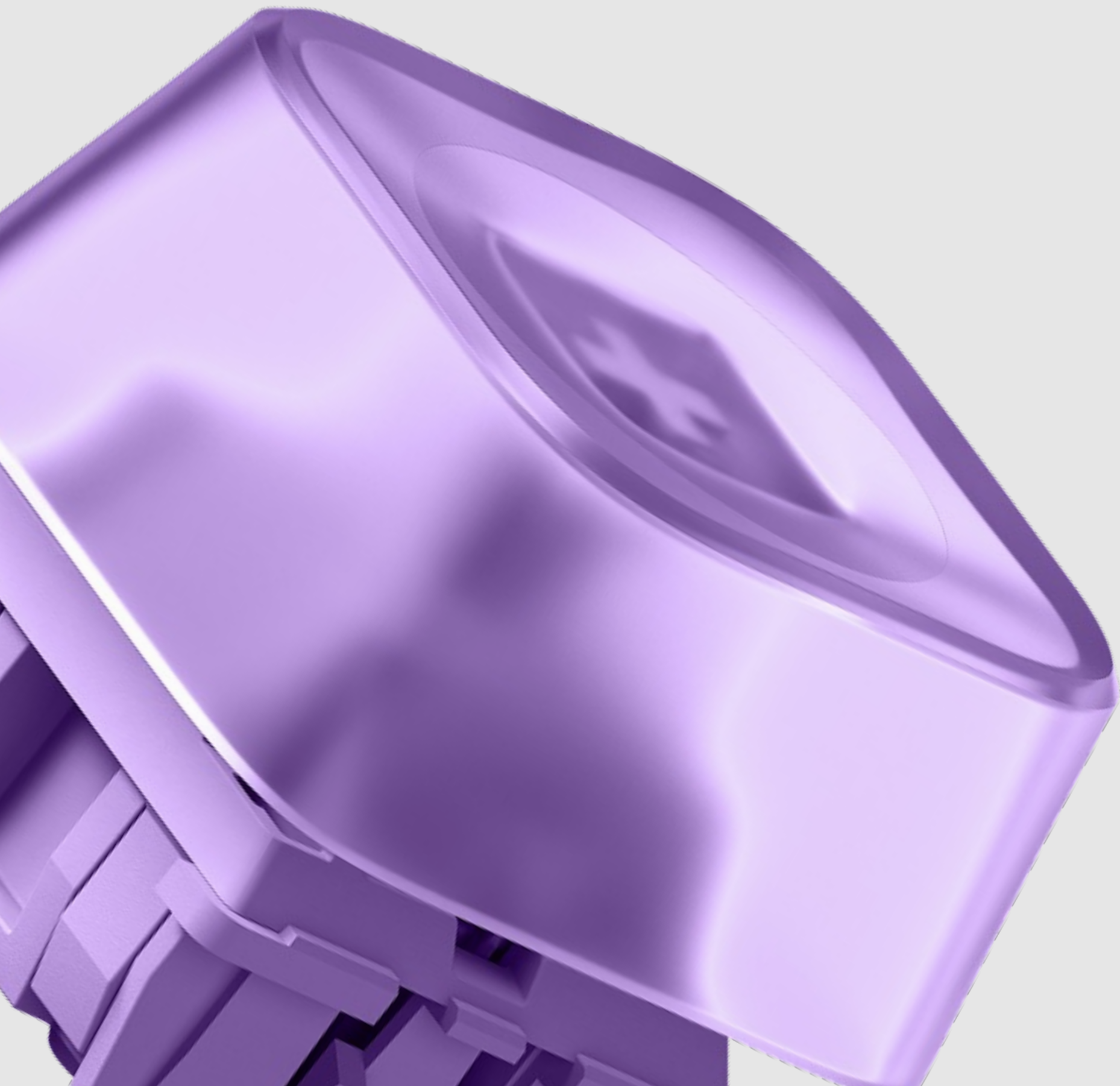


Индексы. Типы индексов

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 9

О чём сегодня поговорим



→ Почему большой SELECT — больно

→ Что такое индекс

→ B-Tree

→ GIN, GIST, BRIN

→ Функциональные индексы

Как PostgreSQL ищет данные по умолчанию

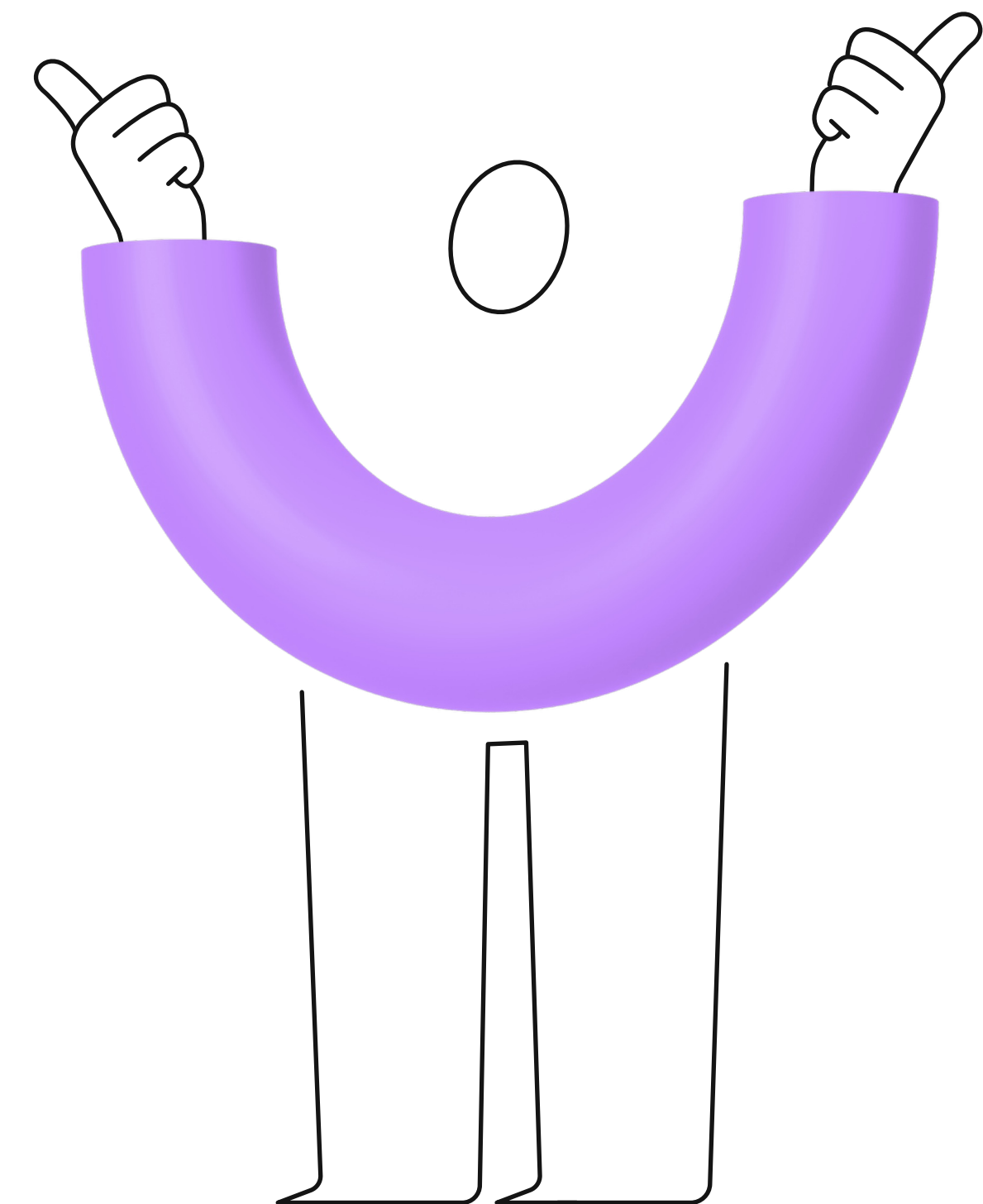
```
SELECT * FROM books WHERE title = 'Война и мир';
```

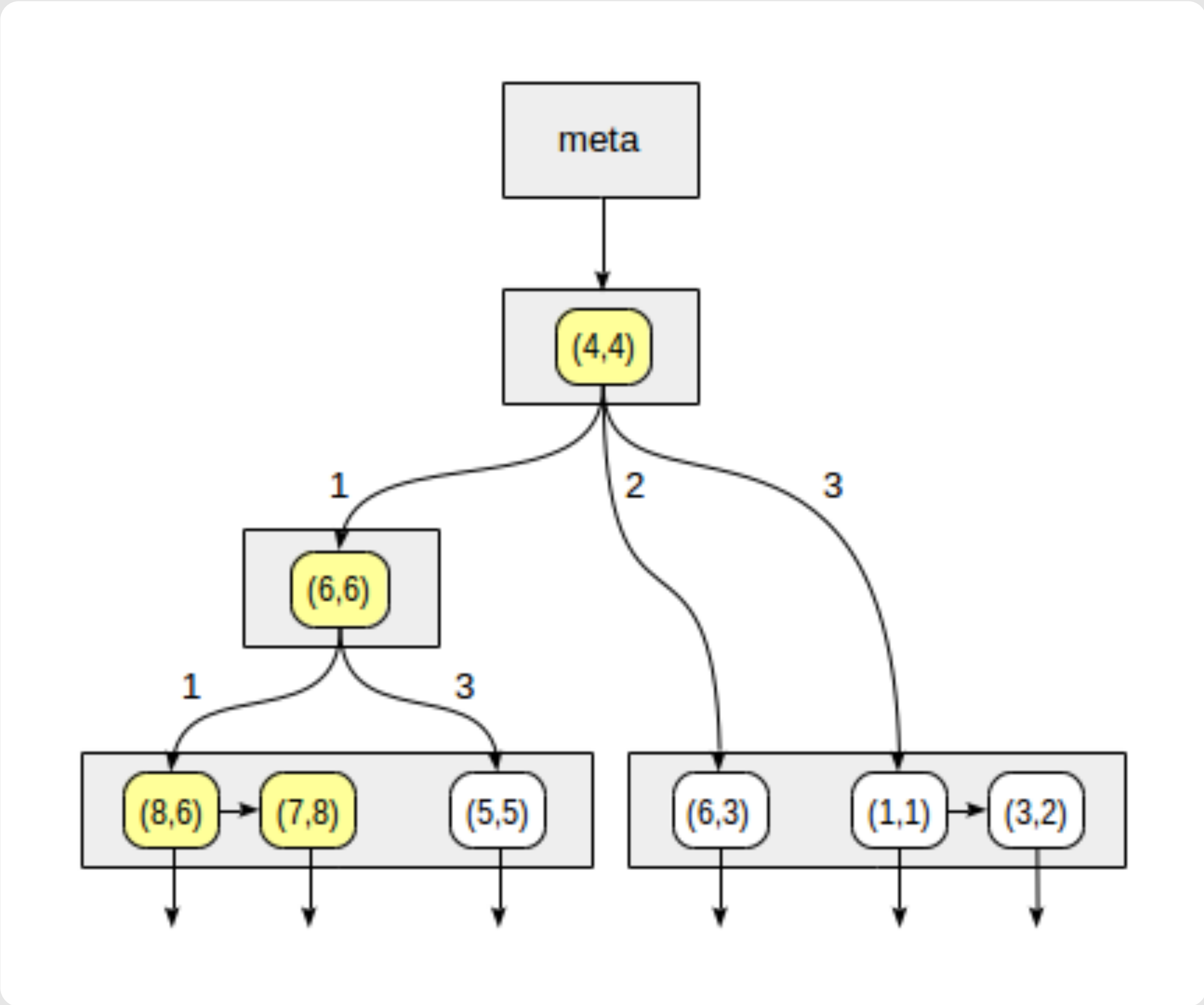


Из чего состоит «медлительность»

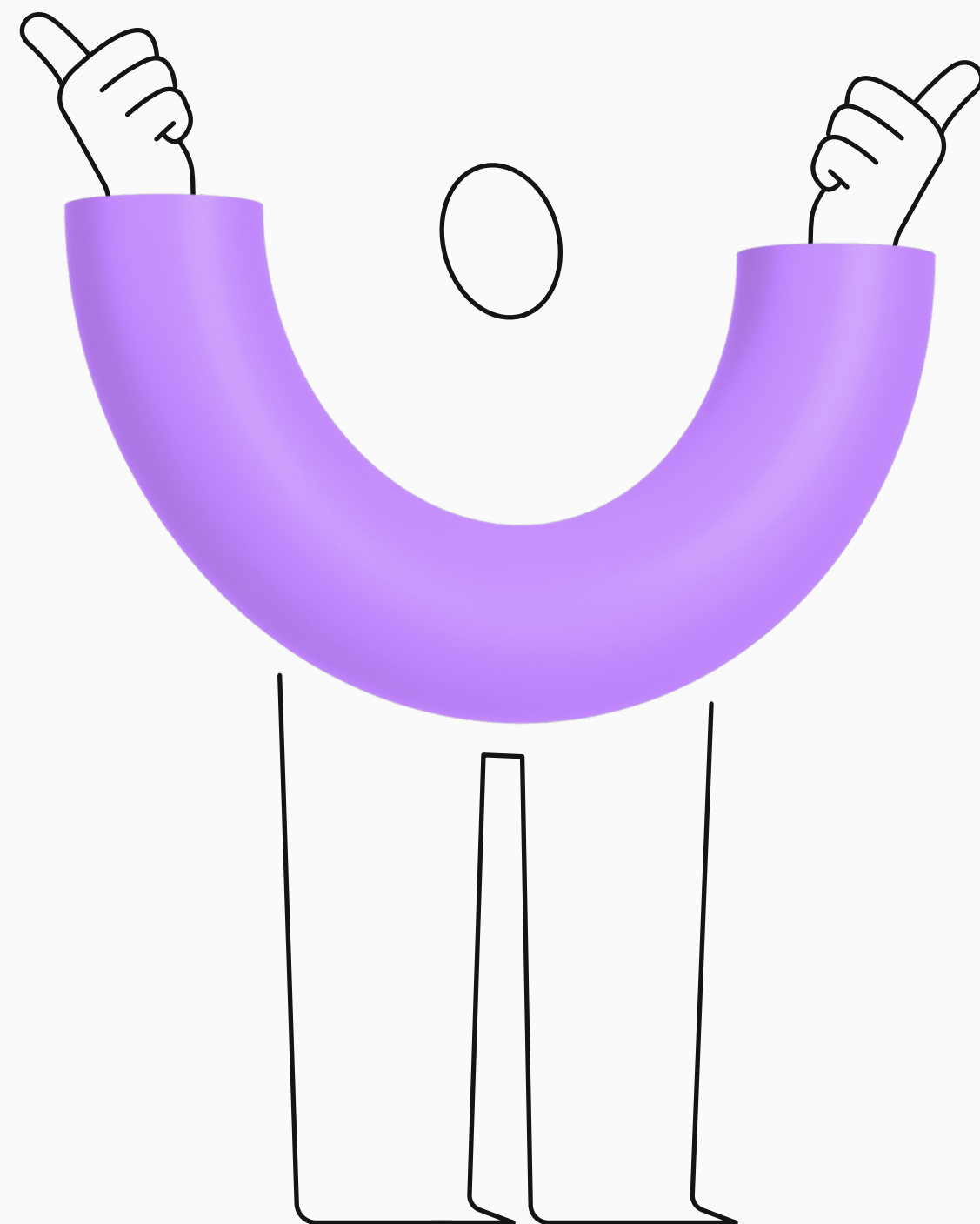
Внутренняя стоимость Seq Scan

- CPU (сравнение данных)
- I/O (чтение с диска)
- RAM (shared_buffers)
- MVCC Visibility Checks & Tuple Deforming (CPU)





Анатомия Index Scan



Index Scan

Быстрый поиск в **индексе** (обычно он маленький и помещается в RAM), получение ctid

Bitmap Heap Scan

«Прыжок» по ctid в **основную таблицу (heap)** для извлечения остальных полей строки

Компромиссы индексов

Плюсы

Ускоряют SELECT

Минусы

Замедляют INSERT, UPDATE, DELETE


```
1 CREATE EXTENSION IF NOT EXISTS pageinspect;
2
3 -- Читаем метаданные первой страницы (листового узла) индекса
4 SELECT btpo_prev, btpo_next, btpo_level, btpo_flags
5 FROM bt_page_stats('users_id_idx', 1);
6
```

Ожидаемый вывод:

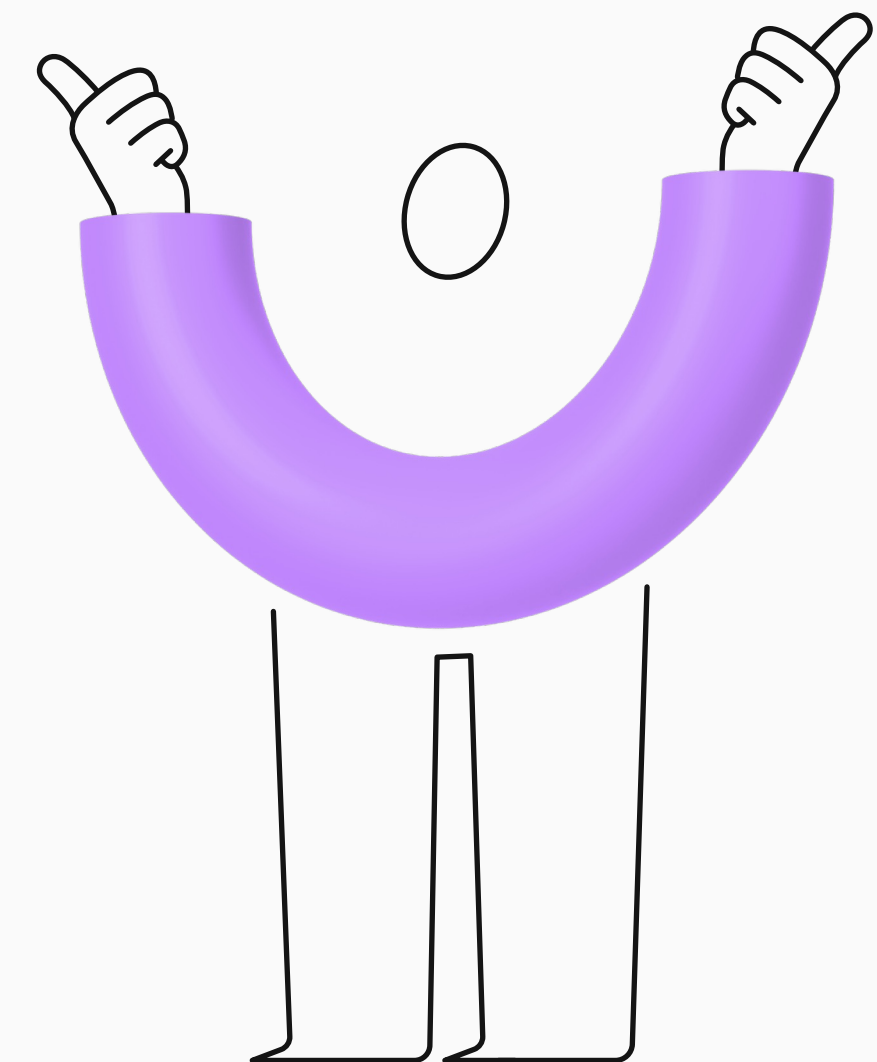
btpo_prev	btpo_next	btpo_level	btpo_flags
0	2	0	1

Важно иметь доступ к системным таблицам, иначе будет следующее.

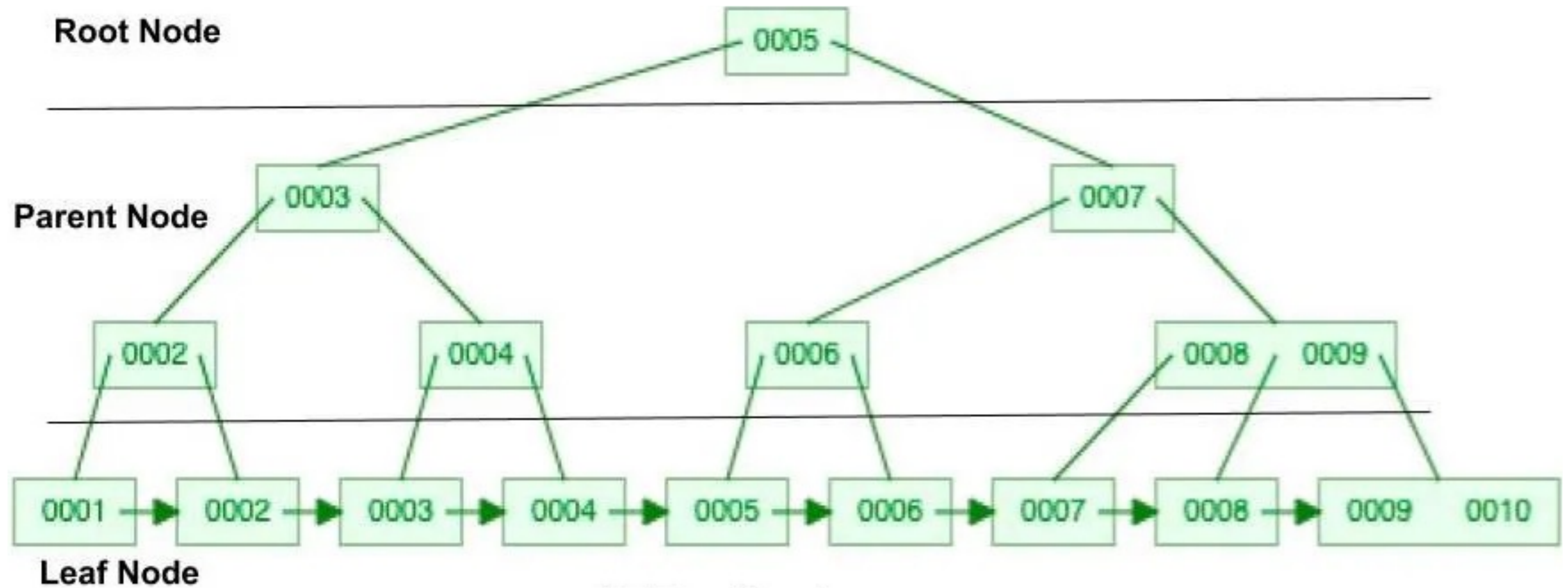
Output:

126 ms

```
psql:commands.sql:1: ERROR: permission denied to create extension "pageinspect"
HINT: Must be superuser to create this extension.
```



Сбалансированное дерево (B-Tree)



```
1  -- Создаем таблицу с резервированием 20% места на каждой странице
2  CREATE TABLE sessions (
3      id serial PRIMARY KEY,
4      user_id int,
5      last_action timestamp
6  ) WITH (fillfactor = 80);
7
8  -- Индекса на last_action НЕТ. Делаем 10 000 апдейтов last_action.
9
10 -- Проверяем статистику:
11 SELECT n_tup_upd, n_tup_hot_upd
12 FROM pg_stat_user_tables WHERE relname = 'sessions';
13
14 -- Ожидаемый вывод:
15 -- n_tup_upd = 10000
16 -- n_tup_hot_upd = 10000
17
```

Как меняется индекс?

***UPDATE**

DELETE + INSERT.

Может вызвать «расщепление» страниц индекса.

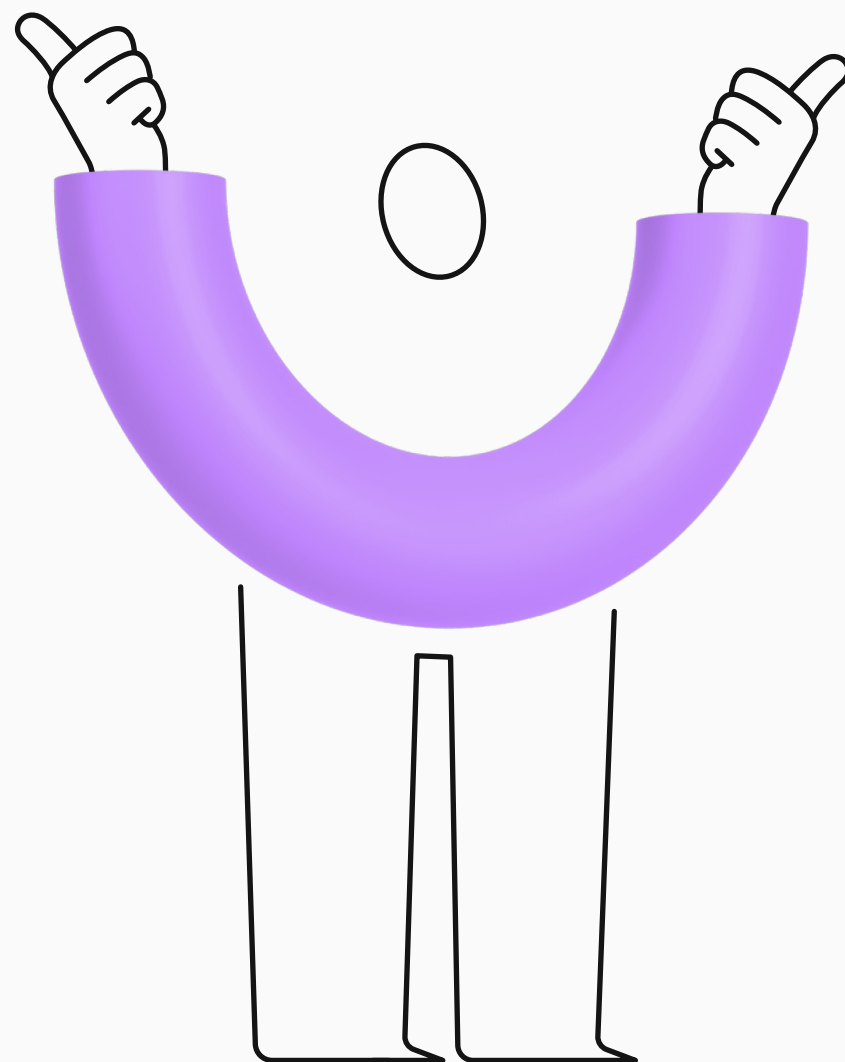
DELETE

Запись в индексе помечается как недействительная.

VACUUM удаляет эту запись во время очистки.

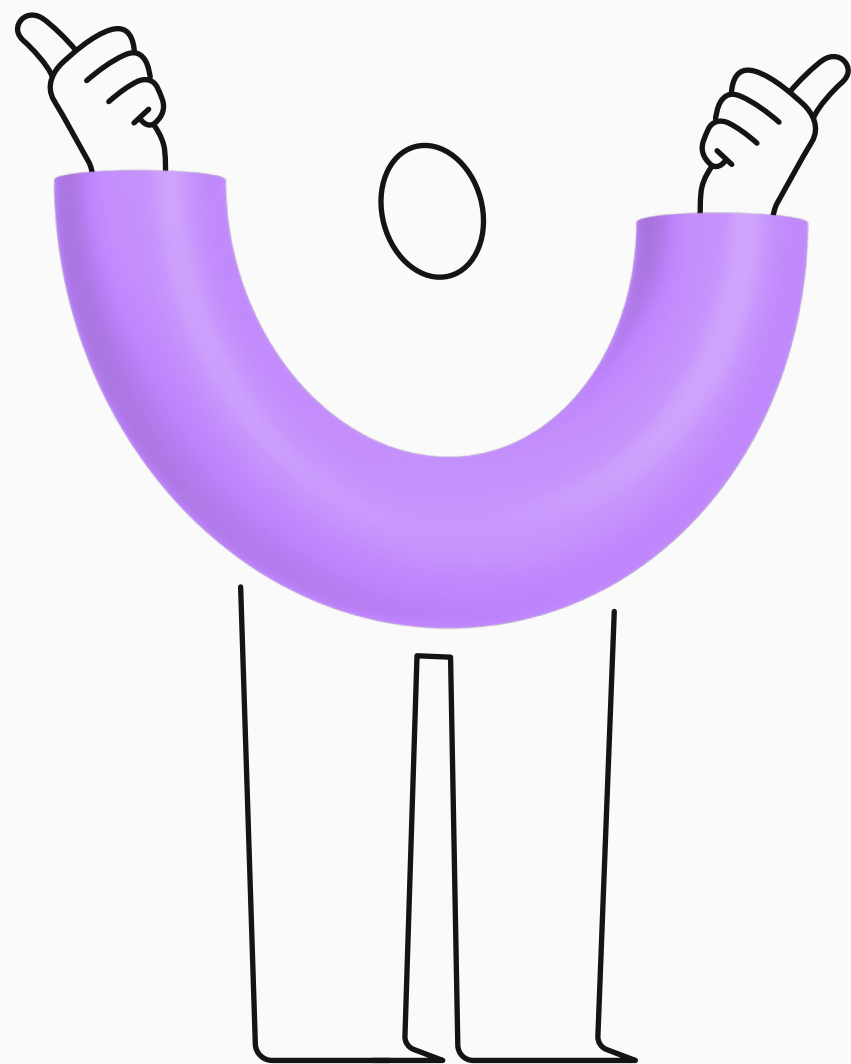
Проблема

Почему создание индекса на «безобидное» поле может замедлить систему в 10 раз.



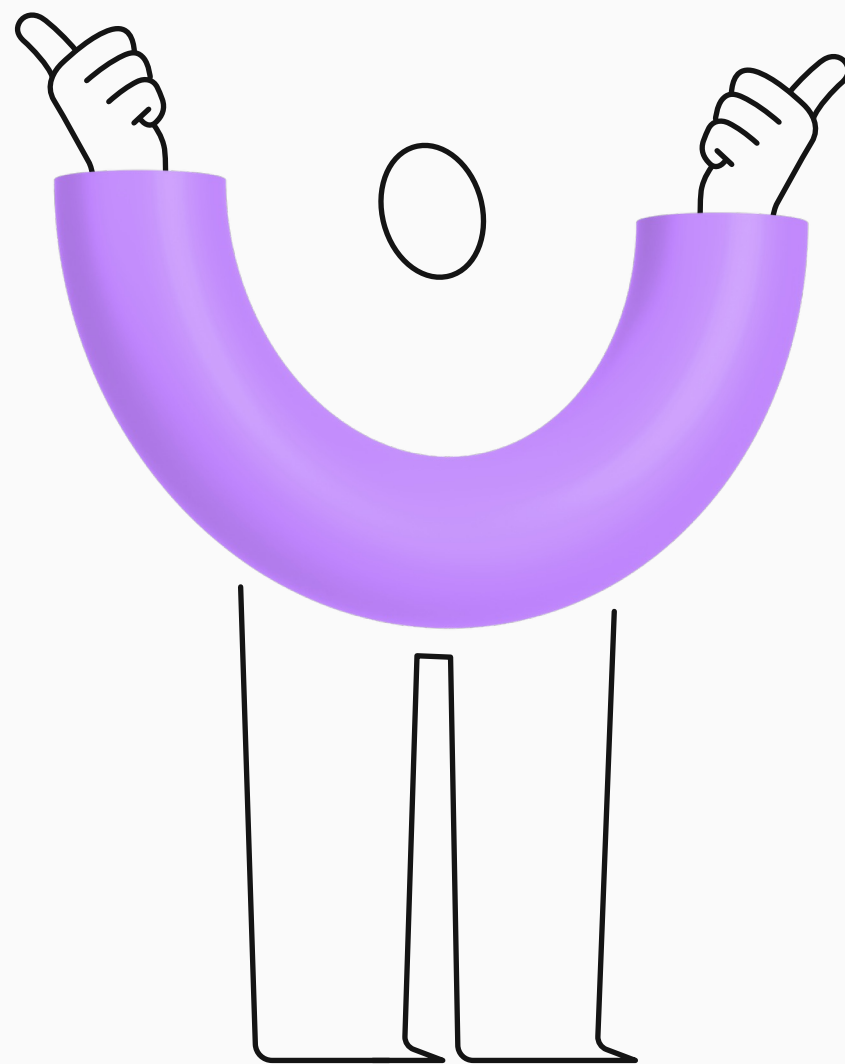
1. Есть таблица users с индексами по email и id. Система часто обновляет поле last_seen (время последнего захода).
2. Работает оптимизация HOT, индексы не трогаются.
3. Добавляется индекс на last_seen.
4. Теперь каждый UPDATE принудительно обновляет записи во всех индексах.

Идеальные сценарии для B-Tree



- Равенство
- Диапазоны
- Сортировка
- Префиксный поиск
- Поддерживает: $=$, $>$, $<$, $\geq$, $\leq$

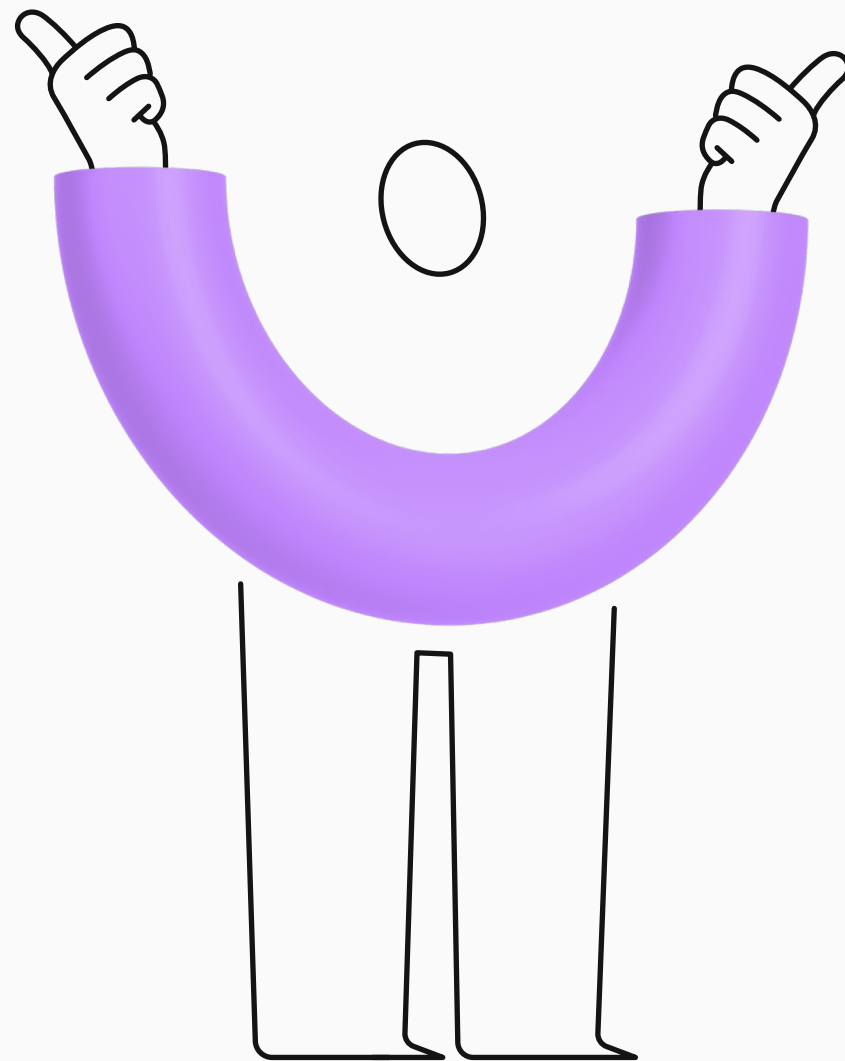
B-Tree и NULL



NULL индексируется в PostgreSQL.

WHERE column IS NULL — эффективный сценарий для B-Tree-индекса.

Составные индексы



Проблема

```
SELECT ... WHERE last_name = 'Иванов' AND  
first_name = 'Иван';
```

Решение

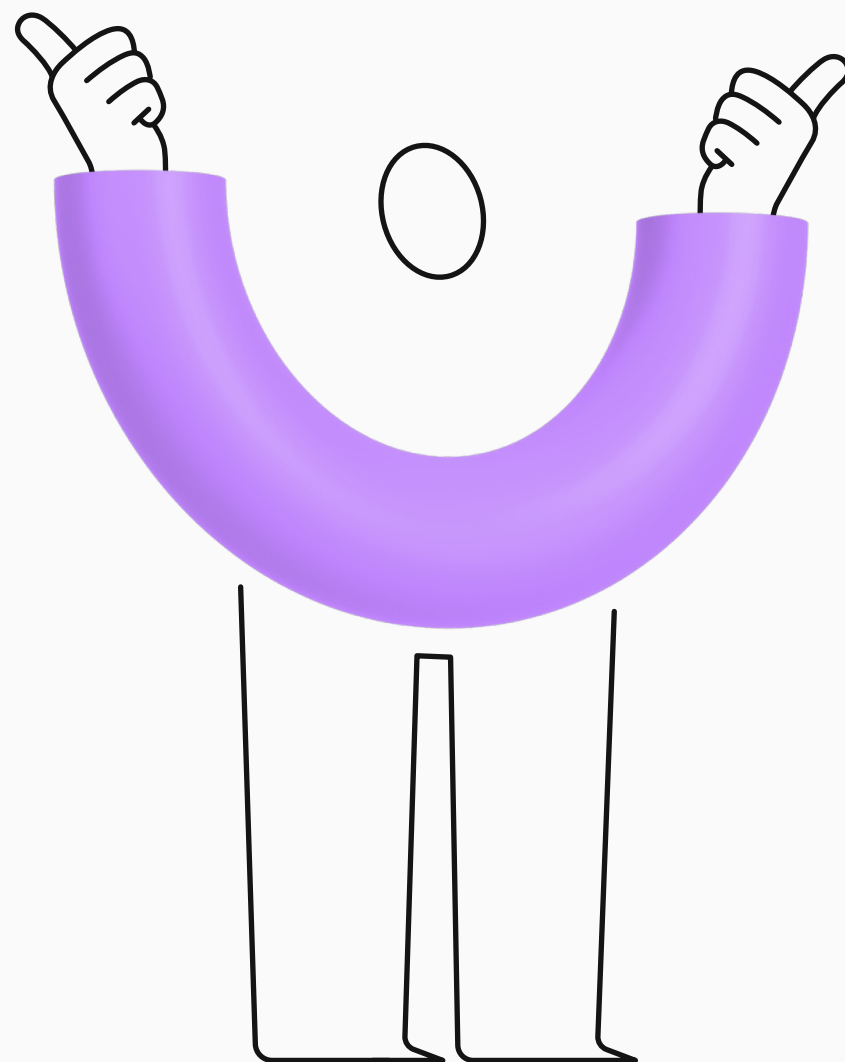
```
CREATE INDEX ON users (last_name, first_name);
```


Почему порядок так важен?

```
CREATE INDEX ON users (city, registration_date);
```

Запрос	Эффективность
WHERE city = 'Москва'	Высокая
WHERE city = 'Москва' AND registration_date > ...	Идеально
WHERE registration_date > ...	Низкая

Планировщик запросов

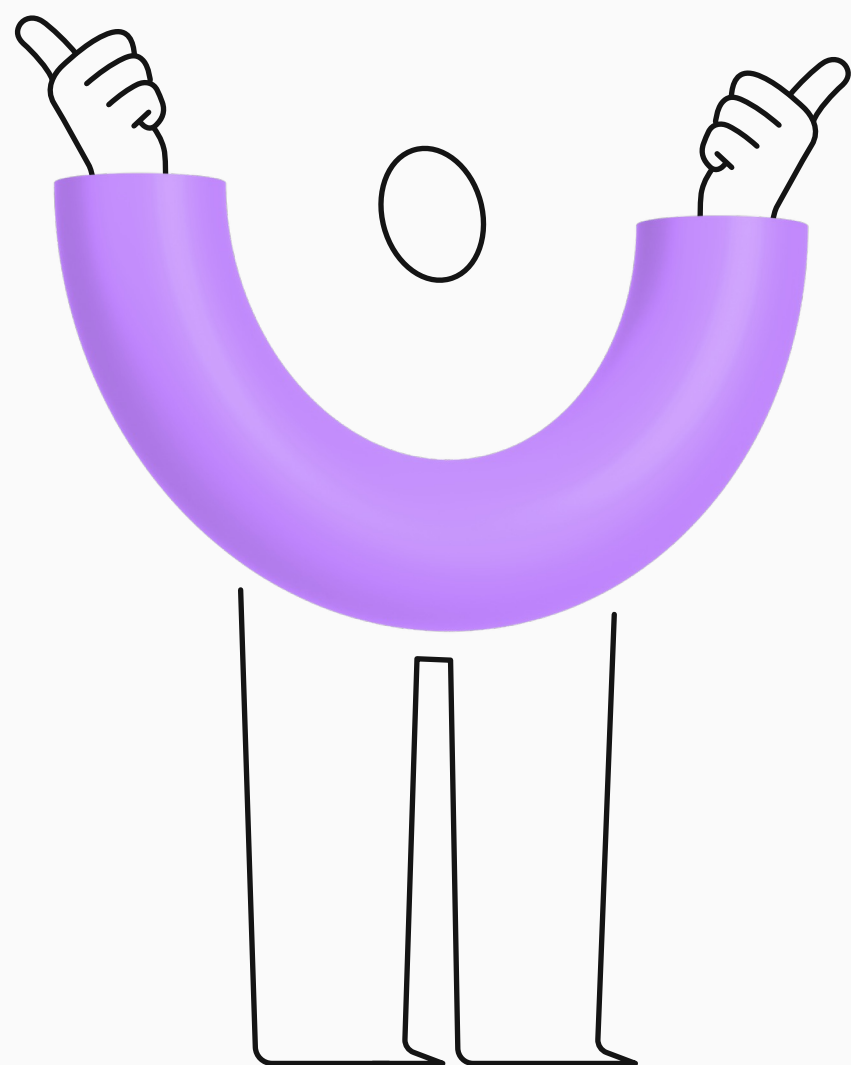


«Мозг» PostgreSQL



Выбрать самый «дешёвый» (быстрый)
план выполнения запроса

Сканирование



Как PostgreSQL читает данные

- Seq Scan
- Index Scan
- Index Only Scan
- Bitmap Scan

Анализ медленного JOIN

SQL-запрос

```
SELECT * FROM users u JOIN orders o  
ON u.id = o.user_id WHERE  
u.registration_date > ...;
```

Решение

```
CREATE INDEX ON users  
(registration_date); CREATE INDEX ON  
orders (user_id);
```

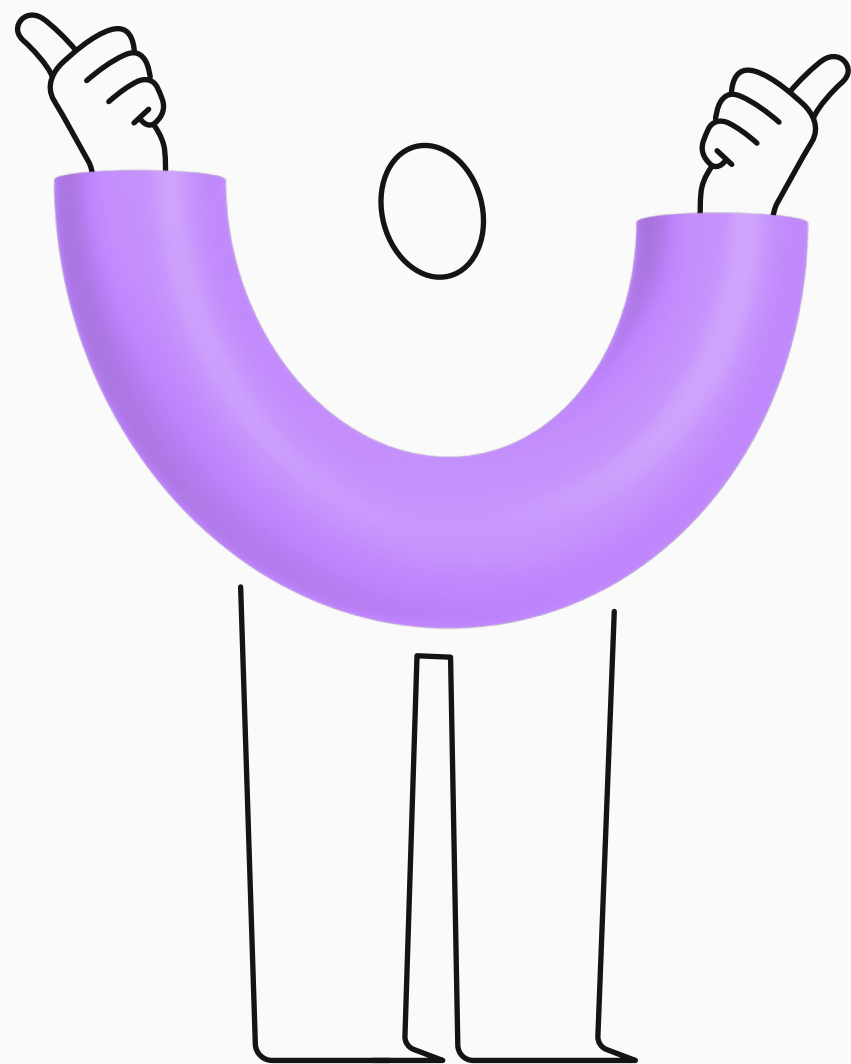
ДО

Hash Join -> Seq Scan on orders
и Seq Scan on users

ПОСЛЕ

Nested Loop Join → Index Scan
on users и Index Scan on orders

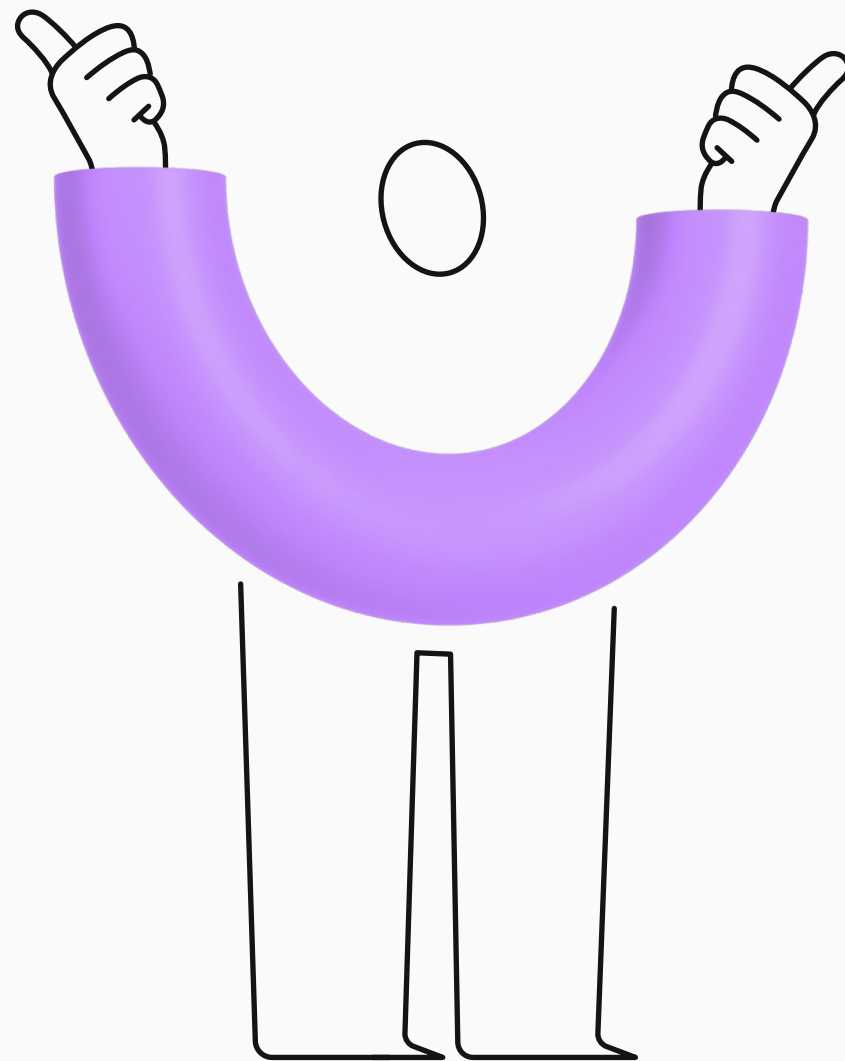
Когда B-Tree не справляется?



Инструменты для особых задач

- Массивы
- JSON
- Полнотекстовый поиск
- Геоданные

Generalized Inverted Index



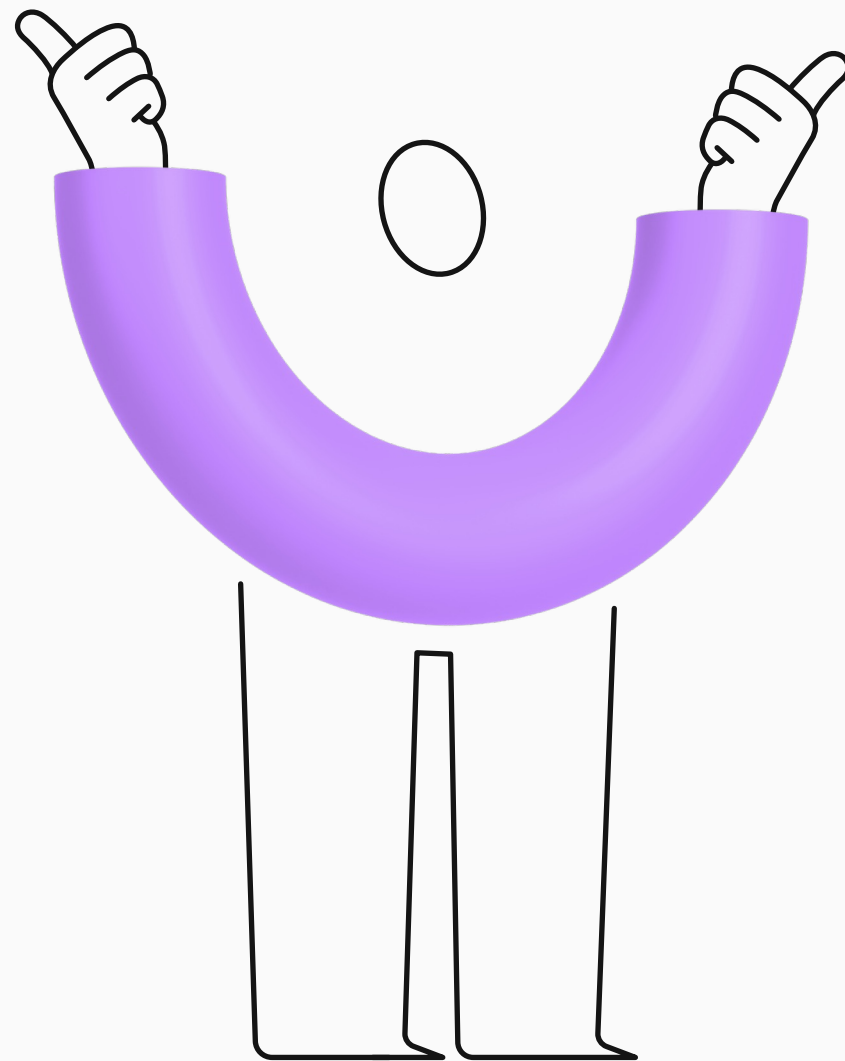
Индекс для «составных» типов

- Массивы
- JSON
- Полнотекстовый поиск

`CREATE INDEX ON events USING GIN (payload);`

`WHERE payload ? 'error_code', WHERE payload @>
'{"user_id": 123}'`

Generalized Search Tree



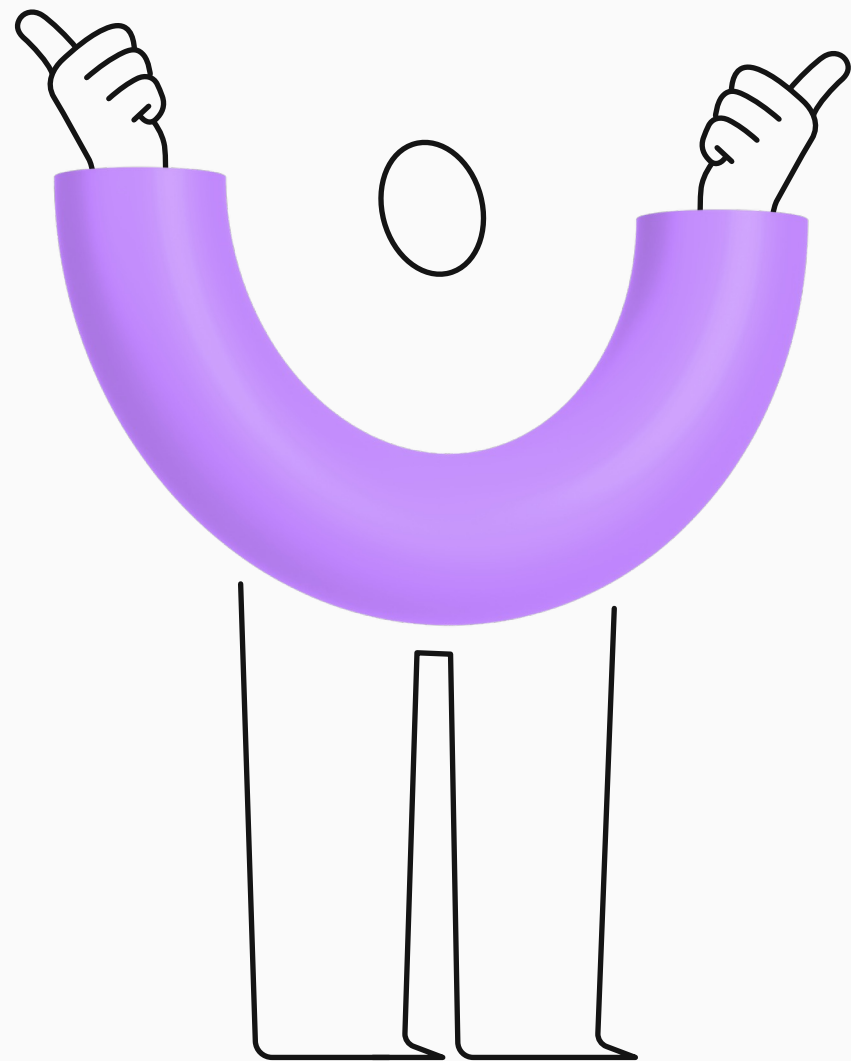
Индекс для «нечётких» данных

- Геометрических типов (PostGIS)
- Поиска по схожести

```
CREATE INDEX ON restaurants USING GIST (location);
```

```
WHERE ST_DWithin(location, ..., 5000)
```

Block Range Index

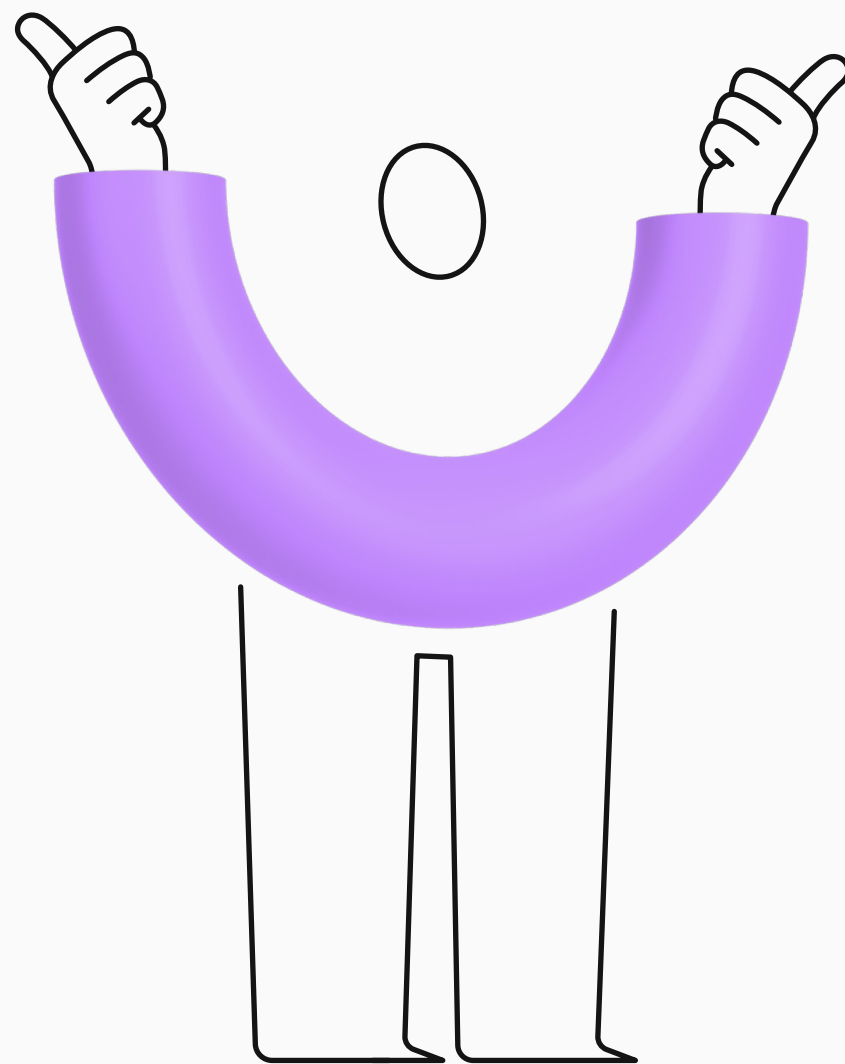


«Микро-индекс» для гигантских таблиц

Идеален для огромных физически упорядоченных таблиц

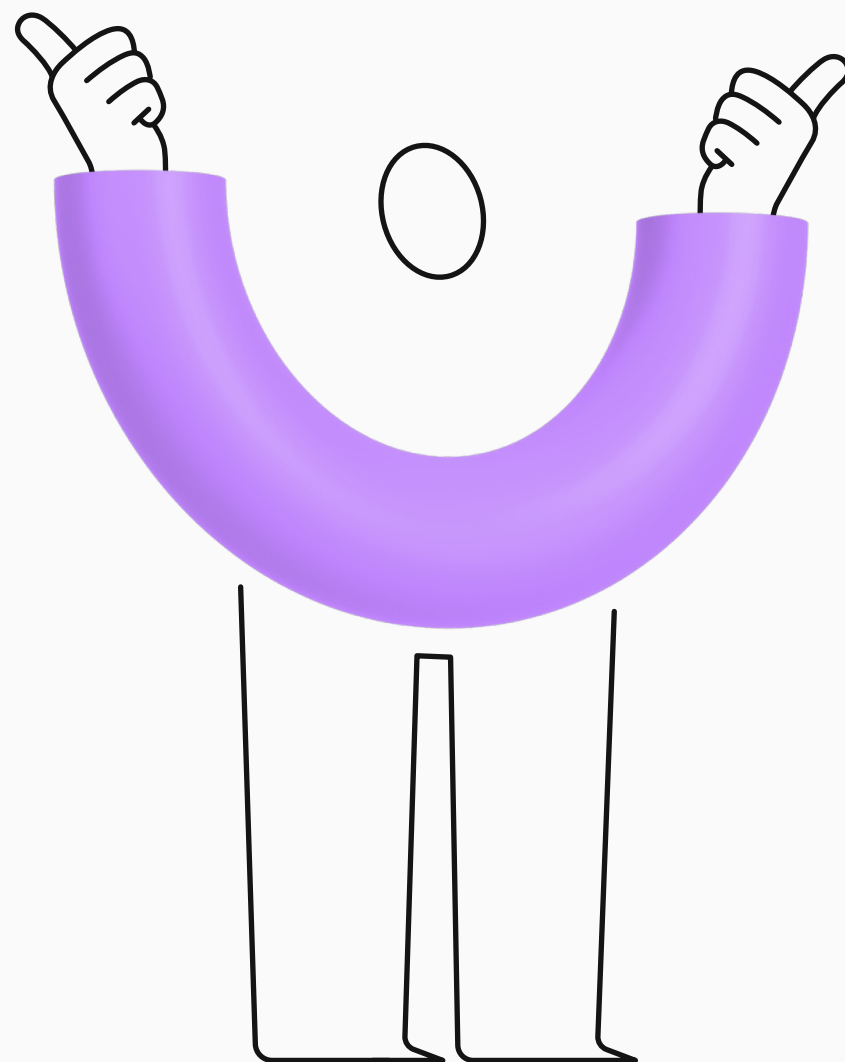
	B-Tree	BRIN
Размер	~25 Гб	~12 Мб
INSERT	Замедление 30–50%	Замедление < 1%
SELECT	0,1 ms	20 ms

Продвинутые техники и архитектура



- Partial
- Covering
- Functional Indexes
- NULL
- Индексы и миграции

Частичные индексы



Индексируем только то, что нужно

Проблема

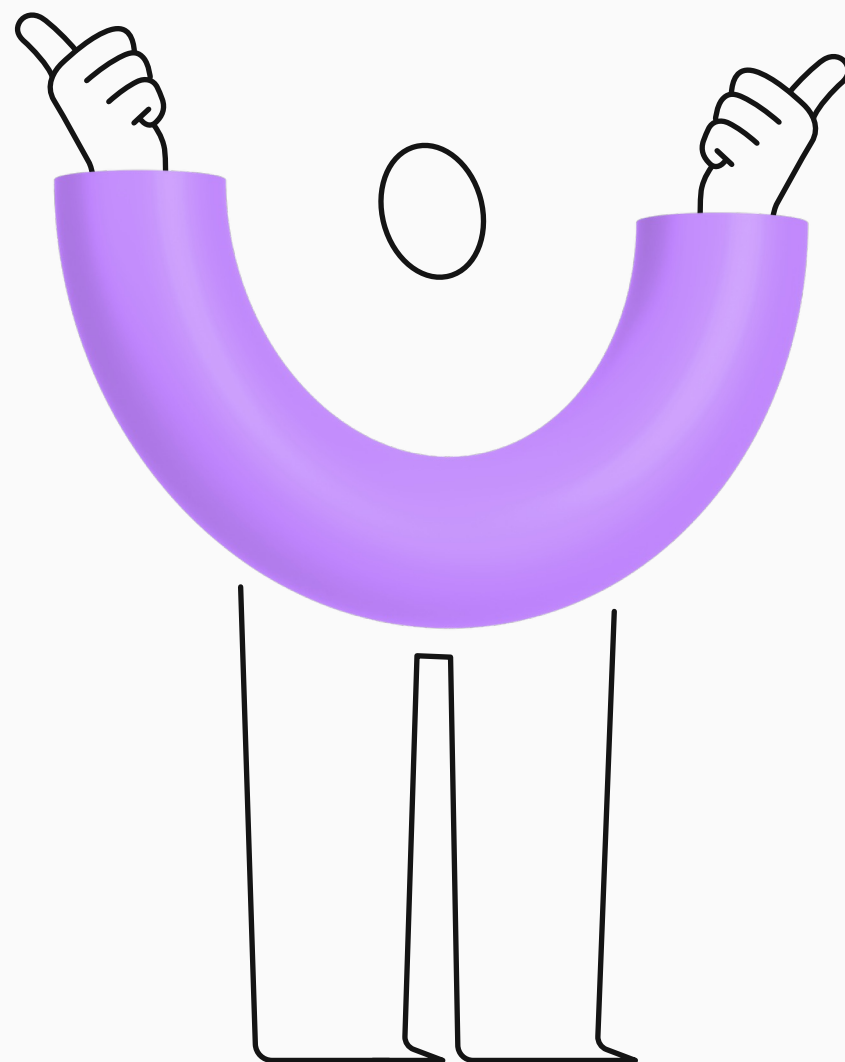
99% заказов completed, ищем только 1% pending

Решение

```
CREATE INDEX ON orders (order_id)  
WHERE status = 'pending';
```

Покрывающие индексы

Index Page → Visibility Map →
Heap Page (опционально)



Избегаем обращения к таблице (Index Only Scan)

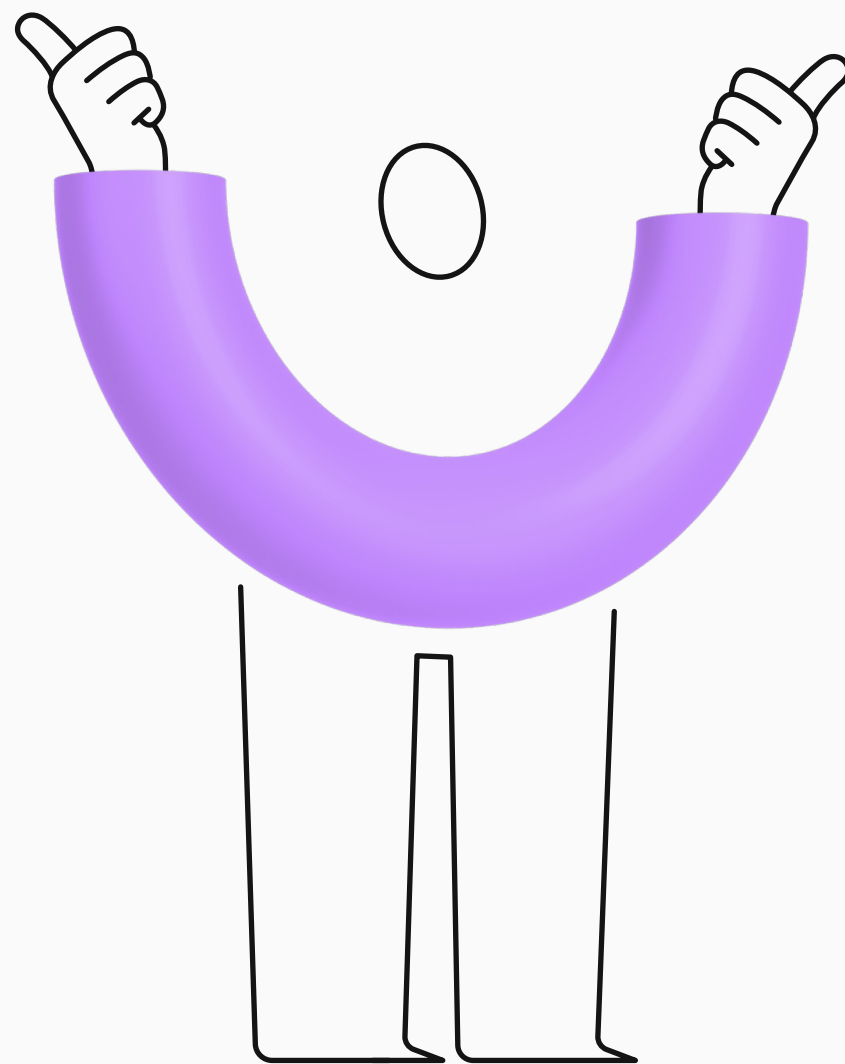
Запрос

```
SELECT email, created_at FROM users WHERE  
user_id = 123;
```

Решение

```
CREATE INDEX ON users (user_id) INCLUDE (email,  
created_at);
```

Индексы по выражениям



Индексируем результат функции

Проблема

```
WHERE LOWER(email) = 'test@example.com';
```

Решение

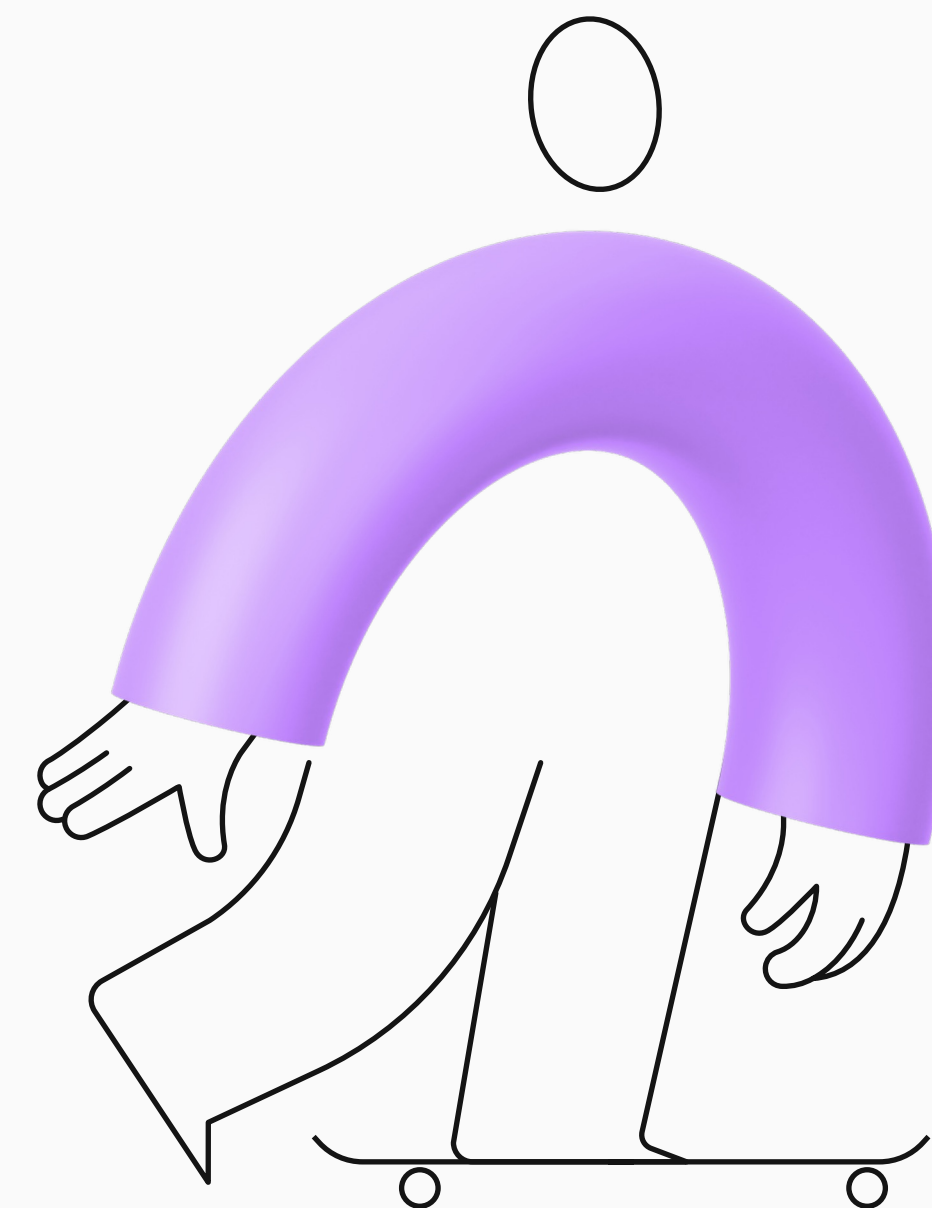
```
CREATE INDEX ON users (LOWER(email));
```


Ловушка OFFSET

```
SELECT * FROM users  
  
ORDER BY registration_date  
DESC,  
  
id DESC LIMIT 10 OFFSET 10000;
```

Что на самом деле делает PostgreSQL

1. Находит все 10010 строк, удовлетворяющих условию.
2. Сортирует их в памяти или с использованием диска.
3. Выбрасывает первые 10000 строк.
4. Возвращает вам оставшиеся 10.



Keyset (Cursor-based) Pagination

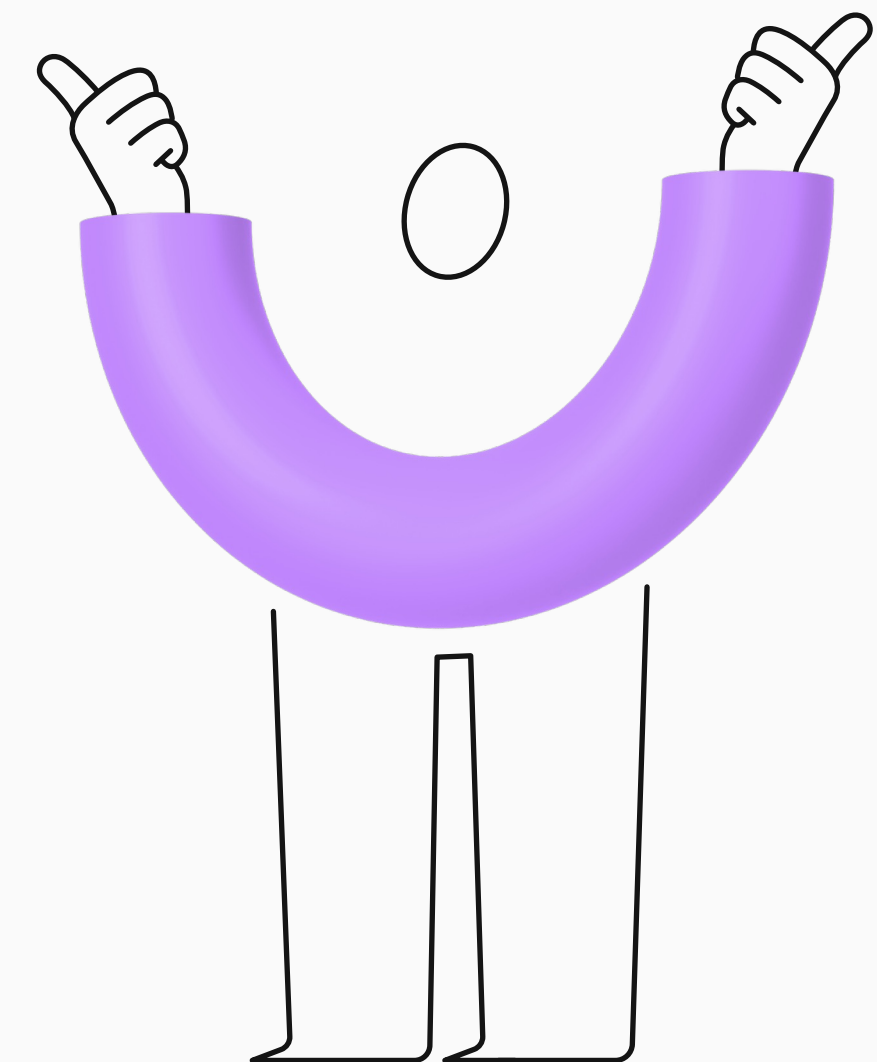
```
SELECT * FROM users  
  
WHERE (reg_date, id) < ('2024-03-15  
10:00:00', 12345)  
  
ORDER BY reg_date DESC, id DESC  
  
LIMIT 10;
```

*Сортировка по (reg_date, id) обязательна

Вместо «пропусти N записей», мы говорим «дай мне N записей, которые идут после вот этой конкретной записи».

Курсор — это и есть «конкретная запись».

Курсором служат значения столбцов сортировки последнего элемента на предыдущей странице.



Keyset Pagination

Как это связано с индексами?

```
WHERE (registration_date, id) < (?, ?)
```

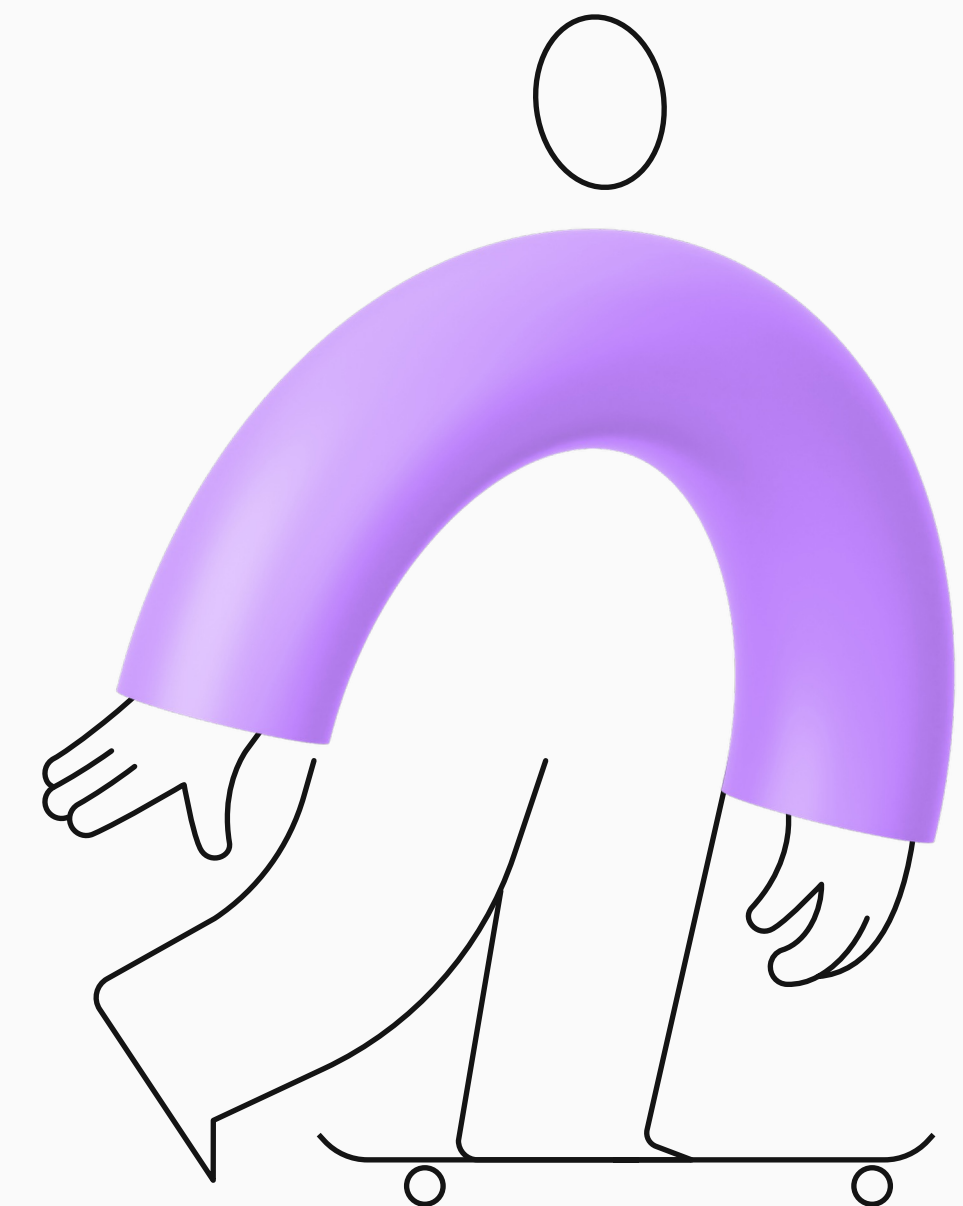
Идеально ложится на составной B-Tree-индекс.

```
CREATE INDEX ON users (registration_date  
DESC, id DESC);
```

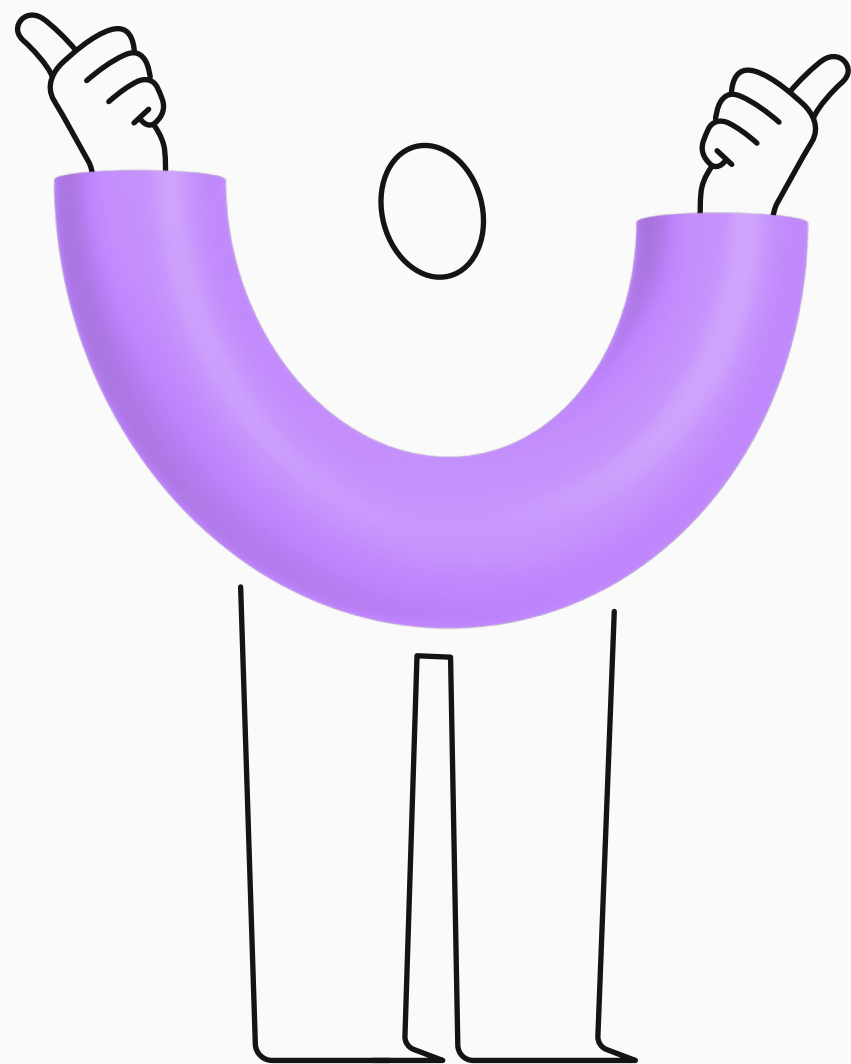
Это простейшая операция —
найти точку в индексе
и читать листья подряд.

Компромиссы и ограничения:

1. нельзя «прыгнуть» на произвольную страницу;
2. требует стабильного и уникального ключа сортировки;
3. клиентская логика усложняется.



Индексы и NULL



Исключаем NULL из индекса

Проблема

Поле `deleted_at`, 99% NULL, ищем только по датам.

Решение

```
CREATE INDEX ON users (deleted_at) WHERE  
deleted_at IS NOT NULL;
```

Индексы и миграции

Фаза 1: Catalog

Фаза 2: Build & Wait

Фаза 3: Catchup & Wait

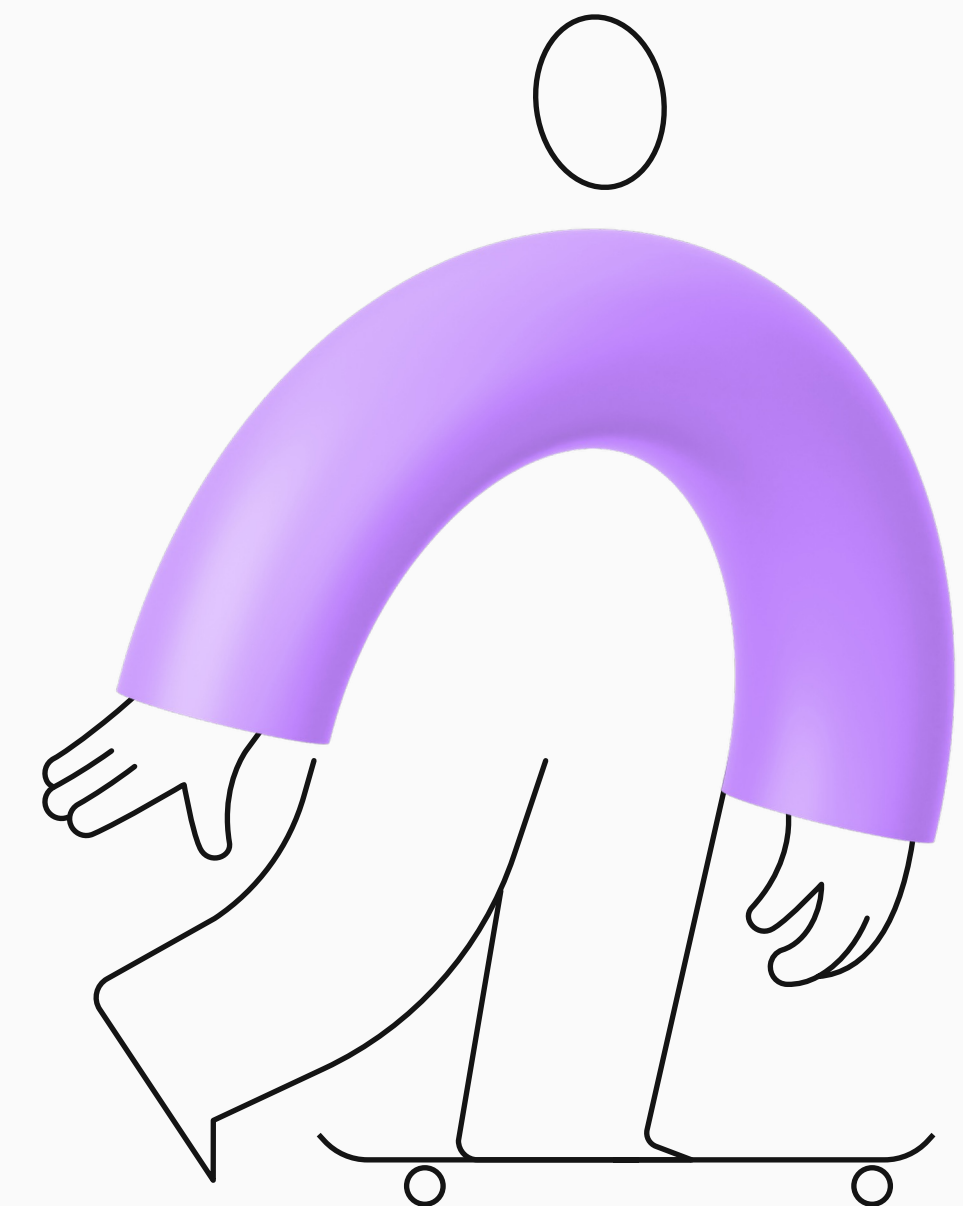
Как безопасно добавлять индексы в Production?

Проблема

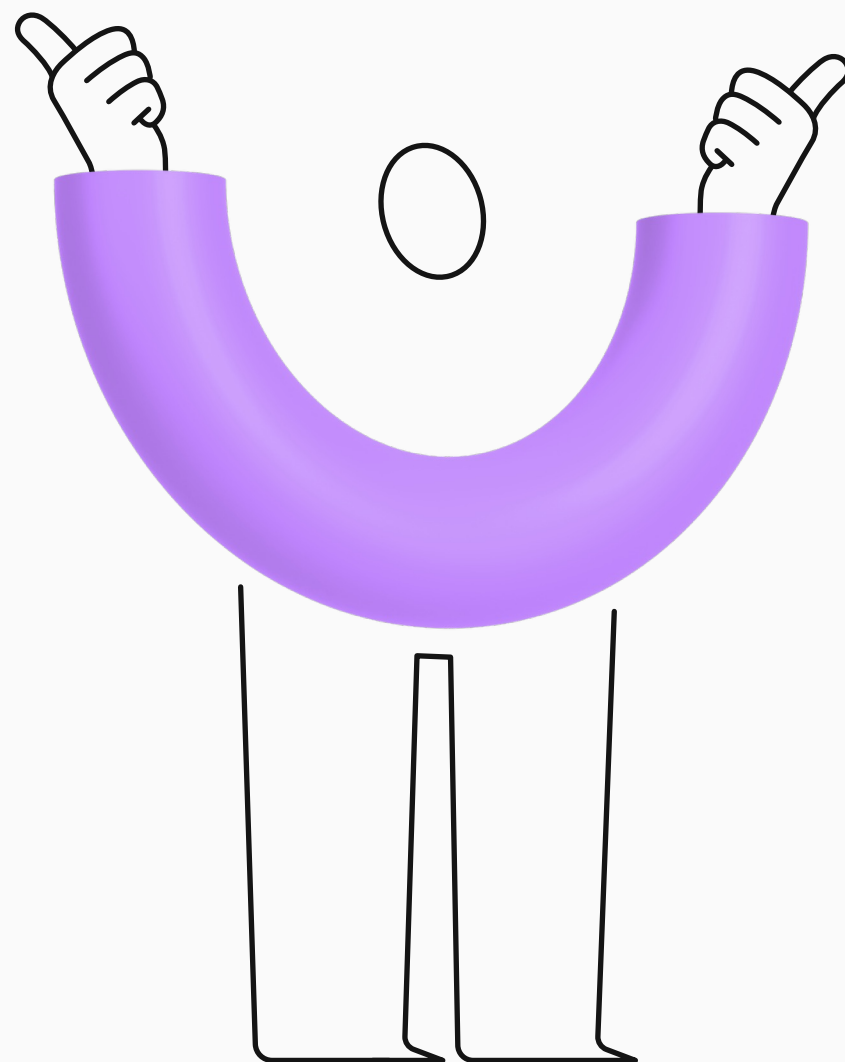
`CREATE INDEX` блокирует таблицу для записи.

Решение

`CREATE INDEX CONCURRENTLY;`



Инструкция: как повесить Production



- Вы запускаете создание индекса в 12:00.
- В это же время в другом соединении открыта транзакция (например, отчёт), которая просто «висит» и ничего не делает.
- SIC обязан дожидаться окончания всех транзакций, которые могли бы изменить данные в таблице. Пока отчёт не закроется, создание индекса не перейдёт к фазе 2.
- Пока SIC висит в ожидании, он может удерживать защёлки на системных каталогах. Другие операции ALTER TABLE или VACUUM встанут в очередь за ним. Начинается «запирание» всей базы.

Какой индекс выбрать?

Индекс	Сильные стороны
B-Tree	Равенство, диапазоны
GIN	Поиск внутри
GiST	Пересечение, близость
BRIN	Огромные упорядоченные таблицы

Best Practices

1. Анализируйте, потом создавайте (EXPLAIN ANALYZE).
2. Индексы эффективны на селективных столбцах (высокая кардинальность).
3. Помните о порядке в составных индексах.
4. Используйте INCLUDE для создания покрывающих индексов.
5. Удаляйте ненужные индексы.



Вопросы

